

Is it ready to charge customers?

Prepared 2026-05-30 · Web (Vercel) + API (Railway) + Postgres/Redis + Stripe + Chrome extension ·
Branch `main` @ commit `1e5a15a`

Verdict: NOT production-ready yet — close on the web app, but four hard blockers gate charging real customers.

The product's **core engine and security fundamentals** are genuinely **solid** (Stripe webhook HMAC + idempotency, no-wildcard credentialed CORS, fail-fast prod boot gates, parameterized SQL, the corrected 104-card data). The gaps are at the **edges that make money and onboard users**: cross-domain auth cookies that Safari silently drops, Stripe price IDs the runbook tells you to set under the wrong names (so checkout fails), a background worker that may not be running, mobile parity gaps, and a Chrome extension that is still an MVP scaffold. None require a rewrite; most are configuration + targeted fixes.

4

LAUNCH BLOCKERS
REMAINING

~20

HIGH-SEVERITY ISSUES

11

FIXED & SHIPPED THIS
SESSION

5

SECURITY
FUNDAMENTALS: SOLID

1 · Executive summary

This assessment combines a live mobile-browser QA sweep of all major routes, a 35-route static responsive audit, a Chrome-extension stress test against the live API, and an infrastructure/deployment review. The findings below separate **FIXED** (done & verified this session) from **BLOCKER** / **HIGH** items that remain.

THE FOUR LAUNCH BLOCKERS (MUST FIX BEFORE CHARGING ANYONE)

1. Stripe price IDs are documented under the wrong names → Pro checkout fails for everyone.

Checkout reads `STRIPE_PRICE_ID_PRO_ANNUAL` / `PROPLUS_ANNUAL` / `LIFETIME`, but `SHIP.md` and `.env.example` only tell you to set `MONTHLY` / `ANNUAL` / `LIFETIME`. Follow the runbook and the default "Pro" checkout throws *"stripe price ID not configured"*. `internal/service/billing.go:84,109,131` · `SHIP.md:38`

2. Cross-domain auth cookies are dropped by Safari / iOS. The app (`vercel.app`) and API (`railway.app`) are different registrable domains, so the API's auth/session/CSRF cookies are *third-party* (`SameSite=None`) and silently blocked by Safari ITP + any "block third-party cookies" setting — a large share of Canadian (iPhone-heavy) users can't stay logged in. The codebase already supports the fix via `COOKIE_DOMAIN`; it needs a shared parent domain (e.g. `app.maplerewards.ca` + `api.maplerewards.ca`), which is a DNS + env change, not a rewrite. `internal/handler/auth_cookies.go:37-47`

3. The background worker may not be running. `cmd/worker` is built into the image but must run as its own Railway service. If it isn't: the PIPEDA 30-day account-deletion purge never executes (a compliance promise), and award-watch alerts, weekly digests, offer-expiry, promo-sentinel and CPP refresh — advertised Pro value — silently never fire. `cmd/worker/main.go` · `Dockerfile`

4. **The Chrome extension does not work in production.** It points at `localhost` + a `maplerewards.app` domain that isn't where prod lives, its marquee "best card on checkout" bar never renders for a normal user, and once a user signs in the extension can't read their (owner-scoped) wallet at all. See §4.
- `browser-extension/*`

2 · What I fixed & shipped this session

All committed to `main` and auto-deployed (Vercel + Railway). Each was verified in a real browser or against the live API.

AREA	FIX	COMMIT
Auth (prod blocker)	Production sign-up/login was failing with "missing or invalid CSRF token" — the SPA couldn't read the API-domain CSRF cookie cross-domain. Now sources the token from the <code>/csrf</code> response body + self-heals on rotation. Verified live: account creation returns 201.	<code>f53637f</code>
Billing UX	After Stripe checkout the profile didn't flip to Pro until the 15-min token expired. Added <code>refreshSession()</code> polling on the success page so Pro reflects within seconds.	<code>d365c4c</code>
Card data integrity	Completed the gated 104-card correction: ~50 earn-rate fixes, BMO eclipse + RBC Rewards+ value-model fixes, Hilton/Marriott transfer dedupes, and deactivation of defunct/fabricated cards (HSBC x3, Capital One x4, etc.). Resolved the 3 flagged items (Desjardins Remises duplicate, PC Money base rate, Odyssey Gold full restructure) + zeroed no-reward cards (Syncro, CIBC Select). DB at migration v86, 0 dupes, 0 transfer conflicts.	<code>059-086</code>
Mobile (blocker)	Chat input bar was hidden behind the bottom nav and off-screen on phones (a <code>position:fixed</code> element trapped by a transformed <code>.reveal</code> ancestor). Fixed the containing block + anchored to the nav height. Verified: input fully visible above the nav.	<code>1e5a15a</code>
Mobile (blocker)	<code>/compare</code> cut card B off-screen and unreachable on phones — now stacks vertically.	<code>1e5a15a</code>
Mobile (systemic)	iOS auto-zoom on every sub-16px input (trip-planner, settings, applications, optimizer, ...) — global rule forces $\geq 16\text{px}$ on phones.	<code>1e5a15a</code>
Mobile	Home card-fan scaled down so it stops clipping off both edges; marketing landing grid collapses to one column; <code>/loyalty</code> value tiles reflow.	<code>1e5a15a</code>
Landing	Cinematic scroll-scrub video wasn't pinning (a <code>body{overflow-x:hidden}</code> killed <code>position:sticky</code>) and didn't load on phones. Fixed the pin, added a mobile autoplay-loop fallback + poster + lighter source.	<code>aec43db</code>
CI	Backend + frontend CI were red — bumped Next 16.1.6→16.2.6 (CVEs), go-redis→v9.6.3 + go 1.25.7 (CVEs), and resolved 38 golangci-lint findings. Both gates green locally.	<code>765d781</code> , <code>76a0b65</code>
Polish	Ask-Maple FAB redesigned as a Siri-style orb; chat "thinking" animation; money-formatting fixes; worker baked into the Docker image.	various

3 · Mobile parity

Status: The worst breaks are fixed and verified; a set of data-dense pages still clip columns on narrow phones. The app never shows a horizontal scrollbar (the scroll root uses `overflow-x:clip`), so the failure mode is "right-hand columns cut off," not "ugly scroll."

FIXED & VERIFIED DONE

- Chat input usable on mobile (was hidden behind nav) · /compare stacks · card-fan fits · iOS-zoom inputs · landing grid · loyalty tiles.
- Swept clean on mobile (no overflow): `/optimizer`, `/wallet`, `/cards`, `/pro-tools`, `/insights` (index), `/pricing`, `/settings`, `/trip-planner`.

REMAINING MOBILE PUNCH-LIST (HIGH) — DATA-DENSE FIXED-PX GRIDS THAT CLIP ON $\leq 430\text{PX}$

Each is a fixed multi-column grid with no mobile breakpoint; the recommended fix is to collapse to a single column or wrap in a horizontal-scroll container below 560–768px.

HIGH Card-fan is mouse-only on touch

`components/editorial/card-fan.tsx:291–301,371` (used on home)

All elevation/selection is `onMouseMove/Enter`; nothing fires on touch, so phone users get no feedback and tapping just navigates to `/cards`.

Fix: add `onTouchStart/End` handlers + a "tap a card" affordance, or auto-rotate focus on small screens.

HIGH Ledger/row grids overflow on phones

`cards/page.tsx:450` · `insights/page.tsx:261,392,446` · `portfolio/page.tsx:221,241` · `components/optimizer-form.tsx:703` · `onboarding/page.tsx:295`

Fixed-px column templates (e.g. `40px 1fr 140px 130px`) exceed a 360–430px viewport, cutting off the right-hand value columns / CTAs.

Fix: add a `@media(max-width:560px)` rule per row collapsing to `1fr` (stack the secondary values) or wrap in `overflow-x:auto`; for the optimizer runners-up, show rank + name + dollar value and tap-to-expand the rest.

Note on testing: the local automation browser clamps to a 500px minimum width, so the verified-clean pages were checked at 500px (mobile breakpoint range) and the $<430\text{px}$ -specific breaks above were identified from the responsive audit of the actual grid definitions. Spot-check the punch-list on a real 390px device after fixing.

4 · Chrome extension — stress-test results

What it's supposed to do (per manifest + store listing): on 25 Canadian retailer checkouts, show a floating "best card for this purchase" bar; in the toolbar popup, show your wallet; warn when a store doesn't take Amex.

Verdict: a well-architected MVP *scaffold* that does not work end-to-end in production. The API contract and the security of its fetch-proxy are correct; the auth/session model and prod host config are not wired for real users.

WHAT ACTUALLY WORKS VERIFIED

- **API parsing is correct.** Against the live API, an anonymous wallet's `GET /wallet/{id}` and `POST /optimize` return exactly the shapes the popup and content script parse (`card.name` / `point_balance` ; `card_name` / `program_name` / `effective_return`). The Amex-blackout warning logic is sound.
- **The background fetch-proxy is production-grade security.** Strict `sender.id` check, empty `externally_connectable` , and an anchored method+path allow-list (only `GET ^/wallet/[a-f0-9]{32}$` and `POST ^/optimize$`) — no IDOR/SSRF surface beyond the two intended calls.
- **Icons are real**, correctly-sized branded PNGs (16/32/48/128) — submission-ready (the README/STORE-LISTING calling them "placeholders" is stale).

WHAT'S BROKEN / BLOCKS PRODUCTION

BLOCKER Can't reach production at all

`manifest.json:8-12` · `background.js:11`

`DEFAULT_API_BASE` is `http://localhost:8080` and `host_permissions` lists only localhost + `maplerewards.app` / `*.maplerewards.app` — none of which is the real prod host (`maplerewards-production.up.railway.app` / `maplerewards.vercel.app`). A published extension can't call the API.

Fix: add the real prod hosts to `host_permissions` and set `apiBase` to the prod API (ideally the future `api.maplerewards.ca`).

BLOCKER The marquee floating bar never renders for a normal user

`content.js:47-55,77-83` · `popup.js:46-75`

The content script reads the session only from `chrome.storage.local.mr_session_id` — which is never populated automatically. The popup's cookie-bridge reads a `mr_session` cookie that **the backend never sets** (verified), then falls back to a value you must paste in by hand via DevTools. So the headline feature requires manual setup that no real user will do.

Fix: build the real session bridge — have the web app `postMessage` the session id to the extension on login (or set a readable cookie on the shared parent domain), and make `content.js` use it.

BLOCKER The access model contradicts the listing the moment you sign in

`content.js` / `popup.js` (cookie-only auth) vs `RequireSessionOwner`

The extension authenticates only via cookies. Once a user signs up, their wallet is owner-scoped and the API returns **404 to any non-authenticated caller** — and the web app's access token lives in memory, not a cookie the extension can send. So the extension only ever works for purely anonymous wallets, while the app funnels everyone into sign-up.

Fix: bridge a real auth token to the extension, or add an extension-specific scoped token endpoint.

Recommendation: treat the extension as *pre-launch*. It is not ready to publish to the Chrome Web Store; ship the web app first, then do the session-bridge + prod-host work as a dedicated follow-up. Do not list it as a live feature when charging customers.

5 · Infrastructure & deployment

BLOCKERS / HIGH

BLOCKER Stripe tier price IDs unset if you follow the runbook → nobody can buy Pro

internal/service/billing.go:84-86,109,131 · SHIP.md:38 · .env.example:51-52

Current checkout sends `pro_annual` / `proplus_annual` / `lifetime` → reads

`STRIPE_PRICE_ID_PRO_ANNUAL` / `PROPLUS_ANNUAL` / `LIFETIME`. The docs only mention `MONTHLY` / `ANNUAL` / `LIFETIME`, so those tier vars stay empty and checkout throws "price ID not configured."

Fix (your action): on Railway set `STRIPE_PRICE_ID_PRO_ANNUAL`, `STRIPE_PRICE_ID_PROPLUS_ANNUAL`, `STRIPE_PRICE_ID_LIFETIME` to the real price IDs (test IDs in sandbox, live IDs at cutover). Also subscribe the webhook to `charge.refunded` so refunders lose Pro. Update SHIP.md/.env.example.

BLOCKER Cross-domain third-party auth cookies (Safari/iOS drop them)

internal/handler/auth_cookies.go:37-47 · csrf.go

vercel.app ↔ railway.app are different sites, so session/refresh/CSRF cookies are third-party

`SameSite=None` and silently blocked by Safari ITP + third-party-cookie blocking. The body-token CSRF fallback still needs the cookie to round-trip.

Fix (your action): put app + API on one parent domain — e.g. `app.maplerewards.ca` (Vercel) + `api.maplerewards.ca` (Railway) — and set `COOKIE_DOMAIN=.maplerewards.ca`. The code already emits `SameSite=Lax; Domain=...` when `COOKIE_DOMAIN` is set, so this is DNS + one env var. (This also fixes the extension cookie-bridge.)

BLOCKER Background worker not confirmed running

cmd/worker/main.go · Dockerfile

If `./worker` isn't a running Railway service: PIPEDA 30-day deletion purge never runs (compliance), and award-watch alerts / digests / offer-expiry / promo-sentinel / CPP refresh never fire.

Fix (your action): add a second Railway service from the same image with start command `./worker` and the same DB/Redis/API-key env; confirm its logs show the sweep loop.

HIGH Redis env footgun + manual migrations

cmd/api/main.go (REDIS_ADDR/PASSWORD) · SHIP.md

Code reads `REDIS_ADDR` / `REDIS_PASSWORD` but Railway hands out `REDIS_URL` — a mismatch silently falls back to localhost and fails to boot. Migrations are fully manual with no startup guard, risking serving against an old schema.

Fix: map `REDIS_ADDR` from Railway's value (or parse `REDIS_URL`); run `migrate up` as a release step and/or add a startup schema-version check.

WHAT'S GENUINELY SOLID **DO NOT TOUCH**

- **Stripe webhook:** fail-closed on missing secret, bounded body, constant-time HMAC-SHA256, two-sided 5-min timestamp tolerance, and a RESERVED→COMPLETED idempotency state machine with stale-orphan reclaim (409 keeps Stripe retrying). `handler/billing.go`
- **CORS:** exactly one configured origin echoed (never `*`), credentials + fixed allow-list, and prod boot refuses empty/ `*` /non-https origins.
- **Boot gates:** `JWT_SECRET` (≥32 chars, non-default), `CORS_ORIGIN`, `STRIPE_WEBHOOK_SECRET`, `REDIS_PASSWORD` all `os.Exit(1)` in production if missing/invalid.
- **Data:** all SQL parameterized; JWT rejects non-HMAC alg; CSRF double-submit; IDOR guards on session-scoped routes.

6 · Path to launch — prioritized checklist

#	ACTION	OWNER	TYPE
1	Set <code>STRIPE_PRICE_ID_PRO_ANNUAL / PROPLUS_ANNUAL / LIFETIME</code> on Railway (test now, live at cutover); add <code>charge.refunded</code> webhook event	You	Config
2	Move app + API onto one parent domain + <code>COOKIE_DOMAIN=.yourdomain.ca</code> (fixes Safari auth <i>and</i> the extension bridge)	You (DNS) + 1 env	Config
3	Stand up the <code>./worker</code> Railway service; confirm the sweep loop logs	You	Config
4	Confirm Redis wiring (<code>REDIS_ADDR</code> vs Railway <code>REDIS_URL</code>); run migrations to v86 in prod; flush prod Redis card/valuation/wallet keys	You	Config
5	Finish the mobile punch-list (§3): card-fan touch + the fixed-px ledger grids on cards/insights/portfolio/optimizer/onboarding	Eng	Code
6	Extension: defer to post-launch. Build the session bridge + prod hosts before publishing; don't advertise it as live yet	Eng	Code
7	End-to-end smoke on a real iPhone + a real test purchase once 1–4 are done	You	QA

Bottom line: the web app is one focused push from launchable. Blockers 1–4 are configuration you control (Stripe vars, a domain, a worker service, Redis/migrations) — no code rewrite. The mobile punch-list and the extension are engineering follow-ups; the extension should not be a launch-day feature. The security and billing-integrity foundations are strong enough to charge on once the edges are wired.